Bag-of-words classifier

Project 2 — Computer Vision and Pattern Recognition

Elisabetta Chisso (IN2000231)

Summer 2026

1 Introduction

This project implements an image classifier based on the Bag of Words (BoW) approach, applied to the 15-scene dataset by Lazebnik et al. (2006). The dataset contains 1,500 training images and 2,985 test images across 15 indoor and outdoor scene categories.

The goal is to build a complete pipeline that extracts local SIFT descriptors from images, builds a visual vocabulary using k-means clustering, and represents each image as a normalised histogram of visual words. Two classifiers are compared: a 1-NN baseline and a linear one-vs-rest SVM.

2 Pipeline

The pipeline consists of five main steps:

- Dataset loading with consistent labels across training and test splits.
- SIFT descriptor extraction from each image.
- Visual vocabulary construction via k-means on training descriptors.
- Image encoding as an L1-normalised histogram of visual words.
- Classification with 1-NN (baseline) and linear one-vs-rest SVM.

2.1 SIFT descriptor extraction

For each image, OpenCV's SIFT detector automatically identifies keypoints and computes a 128-dimensional descriptor for each one. SIFT descriptors are invariant to scale and rotation, which is important for a heterogeneous dataset like 15-scene.

From the training set (1,275 images after the validation split), approximately 640,000 descriptors are extracted in total, averaging around 500 per image.

2.2 Visual vocabulary

All descriptors extracted from the training images are concatenated and clustered using `MiniBatchKMeans`. The resulting k centroids form the visual vocabulary: the "visual words" that represent recurring local patterns across scenes.

The choice of k involves a trade-off: a vocabulary that is too small produces a coarse representation, while one that is too large yields sparse histograms that are harder to compare. For this reason, k is selected using a validation set (see Section 3).

2.3 Encoding and normalisation

Each image is represented as a histogram of k bins: every descriptor is assigned to its nearest visual word (closest centroid), and the number of occurrences of each word is counted. The histogram is then L1-normalised, converting raw counts into relative frequencies that are independent of the number of descriptors in the image.

2.4 Classifiers

1-NN (baseline): a 1-nearest-neighbour classifier using Euclidean distance. Each test image is assigned to the category of the training image whose histogram is most similar. It requires no training phase but is sensitive to noise in the histograms.

Linear SVM one-vs-rest: a binary classifier is trained for each category. To classify a test image, the real-valued outputs of all 15 classifiers are computed and the image is assigned to the class with the highest score. The SVM is more robust because it learns a global decision boundary from all training examples.

3 Vocabulary size selection

15% of the training set was held out as a validation set (225 images) before descriptor extraction, using stratified sampling to preserve class balance. The vocabulary is built exclusively from the remaining 1,275 training images, and the test set is never used during model selection.

Table 1 reports validation accuracy for four values of k. Figure 1 shows the same results as a plot.

Table 1: Validation accuracy for different vocabulary sizes.

k	1-NN (val)	SVM (val)	Note
50	32.9%	36.0%	
100	32.9%	**42.2%**	best
200	30.7%	40.9%	
500	28.0%	40.0%	

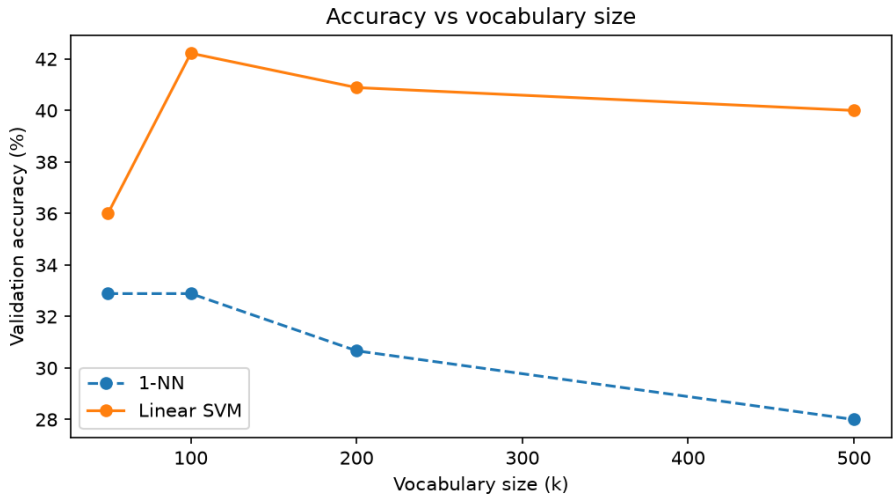

Figure 1: Validation accuracy vs vocabulary size k.

k=100 achieves the best SVM validation accuracy (42.2%) and is therefore selected for the final evaluation. Notably, 1-NN does not benefit from larger vocabularies: as k increases, 1-NN accuracy drops, likely because histograms become sparser and Euclidean distance loses discriminative power. The SVM is more stable across vocabulary sizes, thanks to its regularisation mechanism.

4 Results on the test set

With k=100, both classifiers were retrained on the full training set (1,275 images) and evaluated on the test set (2,985 images). Results are reported in Table 2. Figures 2 and 3 show the confusion matrices on the validation set for the 1-NN and SVM classifiers respectively.

Table 2: Test set accuracy with the selected vocabulary size k=100.

Classifier	Test accuracy
1-NN	36.0%
Linear SVM	**42.3%**

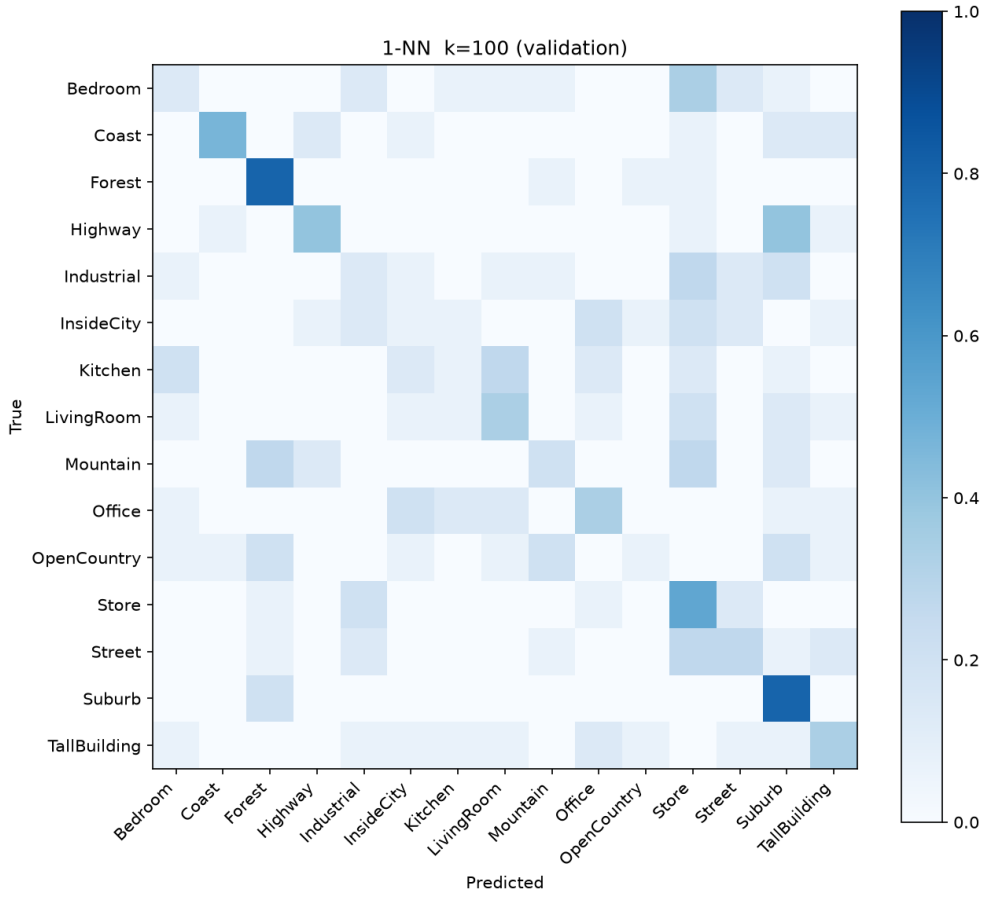

Figure 2: Confusion matrix — 1-NN, k=100 (validation set).

The SVM outperforms the 1-NN baseline by approximately 6 percentage points. This result is consistent with expectations for a standard BoW pipeline on a moderately difficult dataset (a random classifier would achieve around 6.7% on 15 classes).

Inspecting the confusion matrices, the most commonly confused categories are those that share similar low-level patch textures. Open outdoor scenes (Coast, OpenCountry, Mountain) tend to

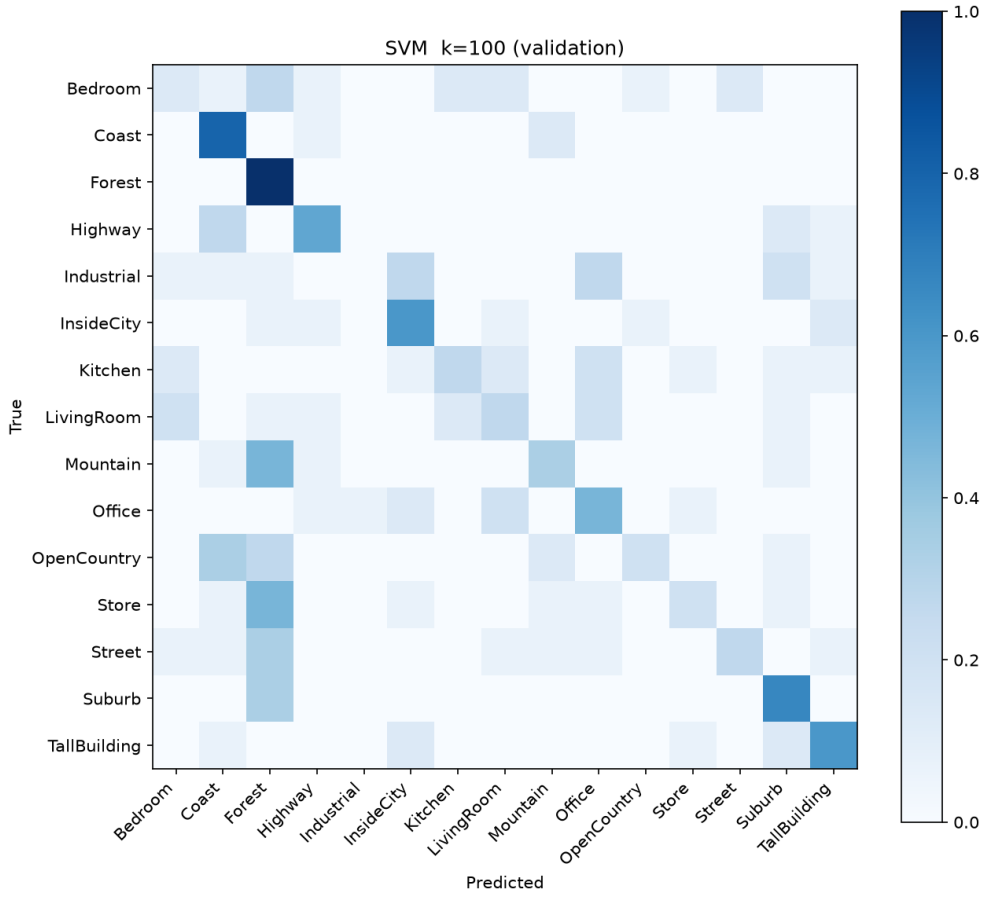

Figure 3: Confusion matrix — Linear SVM, k=100 (validation set).

be confused with each other because they contain large uniform regions (sky, low vegetation, ground). Similarly, indoor categories (Office, Kitchen, Store) are confused due to rectangular objects and regular textures. These errors are expected in a BoW system that discards all spatial information.

5 Conclusions

The pipeline is intentionally minimal and transparent: no descriptor caching, no advanced normalisation, no off-the-shelf BoW library. Each step is implemented with small, well-documented functions that can be read and verified line by line.

The final result (42.3% with linear SVM, k=100) is in line with expectations for a standard BoW baseline. The main limitations are the lack of spatial information and the loss of structure during the pooling step (the histogram does not encode where in the image each visual word appears). Natural extensions would include Spatial Pyramid Matching (Lazebnik et al., 2006) or a chi-squared kernel for the SVM.

Appendix: AI usage

Claude (Anthropic) was used as a support tool during the development of this project. Specifically:

- **Code structure:** Claude suggested organising the code into separate modules (`features.py`, `vocabulary.py`, `classifier.py`, `utils.py`) and helped identify and fix several bugs in

the pipeline (non-deterministic category ordering, descriptor/label misalignment after the train/validation split, figure save paths).

- **Comments and documentation:** Claude helped refine code comments and notebook Markdown cells to make them clearer and more readable for someone unfamiliar with the project.

All implementation choices and experimental results were independently verified and understood. The code was run locally and all reported results are real.